

DocuTrust PKI Architecture Audit

Phased Refactor Plan (Audit Only)

Project: docuTrust | **Document type:** Architecture audit & migration planning | **Status:** No code changes — planning only
Generated: May 14, 2026 | **Scope:** E-signature PKI, sealing pipeline, verification, blockchain anchoring

Executive summary. DocuTrust implements a **two-stage signing model**: field capture (synchronous UI) and cryptographic sealing (async on document completion). PKI is an **embedded application CA**, not a standalone enterprise RCA. This document maps dependencies, risks, and a safe phased refactor toward external RCA/HSM without breaking existing documents.

Table of Contents

1. Current Architecture
2. Dependency Map
3. Service Interaction Map
4. Tightly Coupled Services
5. Migration Risks
6. Signing Flow Risks
7. Blockchain Dependencies
8. Certificate Dependencies
9. Refactor Order (Phases 0–7)
10. High-Risk Areas
11. Migration Strategy
12. Backward Compatibility Strategy
13. RCA Compliance Gap (Procurement Context)

1. Current Architecture

Two-stage signing model

Stage	When	What happens
Capture	Signer submits fields	Keys generated, signer cert issued, <code>Signature</code> row created with <code>image/value</code> but <code>signature_value</code> / <code>signature_hash</code> = <code>null</code>
Seal	Document completes (async job)	Final PDF stamped → SHA-256 hash → per-signer cryptographic seal → blockchain anchor

Signing backends

Backend	Config	Behavior
App-managed	<code>docustrust.pki.signing_backend = app_managed</code>	Internal root CA issues signer certs; keys in encrypted DB columns
Remote-managed	<code>signing_backend = remote_managed</code>	CSC/TSP signs hash; provider certs stored with <code>issuer_certificate_pem</code>

Key source files

Area	Primary files
Root CA	<code>CertificateAuthorityService.php</code> , <code>CertificateAuthority.php</code>
Signer certs	<code>SignerCertificateService.php</code> , <code>SignerCertificate.php</code>
Field capture	<code>FieldSignatureCaptureService.php</code> , <code>SignDocumentController.php</code>
Sealing	<code>CompletedDocumentSealingService.php</code> , <code>SignerSealProviderManager.php</code>
Verification	<code>CertificateVerificationService.php</code> , <code>CertificateChainValidator.php</code>
Blockchain	<code>DocumentHashService.php</code> , <code>BlockchainProofService.php</code>

2. Dependency Map

Layer A — Configuration & DI

Dependency	Consumers	Risk if changed
<code>config('docustrust.pki.*')</code>	CA service, seal manager, signing checks	Global behavior flip
<code>config('services.remote_signing.*')</code>	Remote seal, CSC client, timestamp validator	Remote signing outage
<code>config('services.blockchain.*')</code>	<code>BlockchainProofService</code>	Anchoring silently skipped if empty
<code>AppServiceProvider</code> bindings	<code>CertificateAuthorityKeyStore</code> , <code>SignerKeyStore</code>	Key access path changes

Layer B — Persistence

Table / model	Role	Dependents
<code>certificate_authorities</code>	Self-signed root CA	Signer certs, chain validation
<code>signer_certificates</code>	End-entity + provider certs	<code>signatures</code> , verify UI
<code>document_signers.signing*_key</code>	App-managed keys (encrypted private)	Capture + app seal
<code>signatures</code>	Field data + sealed crypto	Verification, PDF stamping
<code>document_hashes</code>	SHA-256 + <code>transaction_id</code>	Public verify, notary workflow

Layer C — Service dependency tree

```
CertificateAuthorityService
└─ CertificateAuthorityKeyStore (FileBacked → Database fallback)

SignerCertificateService
├─ CertificateAuthorityService
├─ CertificateAuthorityKeyStore
└─ SignerKeyStore

FieldSignatureCaptureService
├─ SignerKeyStore, PkiSignatureService
├─ SignerCertificateService ← issues cert at capture
├─ SigningMethodService
└─ TrustAuthorizationSessionService

SignerSealProviderManager
├─ AppManagedSignerSealProvider
└─ RemoteManagedSignerSealProvider → RemoteSigningClient

CompletedDocumentSealingService
├─ DocumentPdfStampingService
├─ DocumentHashService → BlockchainProofService
└─ SignerSealProviderManager

CertificateVerificationService
├─ PkiSignatureService
├─ CertificateChainValidator
└─ TimestampEvidenceValidator
```

Layer D — External systems

System	Integration	PKI coupling
OpenSSL (PHP)	CA issuance, chain validation, signing	Hard dependency
OpenSSL CLI	TimestampEvidenceValidator	Remote timestamp verify only
Remote CSC/TSP	RemoteSigningClient	Provider-managed certs
blockchain-service	/anchor , /verify	Document hash only — not cert signing

3. Service Interaction Map

Signing path (field-based, happy path)

```
1. User → SignDocumentController → FieldSignatureCaptureService.capture()
2. Capture → SignerCertificateService.getOrIssueForSigner() [cert issued; signature_value=null]
3. Controller → DocumentSigningWorkflowService.completeSignerIfAllFieldsSigned()
4. On all signers done → DocumentStatus::Completed
5. GenerateCertificateJob (async) → CompletedDocumentSealingService.seal()
6. Seal → stamp final PDF → SHA-256 hash
7. Seal → SignerSealProviderManager.seal() per signer
8. DocumentHashService.createOrRefreshForCompletedDocument() → BlockchainProofService.anchor()
```

Verification path

```
verify.blade.php
→ DocumentHashService.verifyStoredProof(documentHash) [blockchain]
→ CertificateVerificationService.verifyDocumentSignatures() [PKI]
  → CertificateChainValidator.validate(certificate)
  → PkiSignatureService.verifySignature()
  → TimestampEvidenceValidator (remote_managed + timestamp)
```

Dual certificate sources

certificate_source	Issuer stored in	Validation path
app_managed (default)	certificate_authorities FK	validateAppManagedCertificateChain
provider_managed	issuer_certificate_pem on row	validateProviderManagedCertificateChain

4. Tightly Coupled Services

Coupling	Why it matters
SignerCertificateService ↔ CertificateAuthorityService	Single root CA, inline OpenSSL; no CA abstraction
FieldSignatureCaptureService ↔ cert issuance	Certs at field save; remote path still generates local keys at capture
CompletedDocumentSealingService ↔ DocumentHashService	Hash + persist + blockchain anchor in one chain
SignerSealProviderManager ↔ global config	One signing_backend for entire deployment
CertificateChainValidator ↔ OpenSSL + DB	Two branches keyed on certificate_source string
GenerateCertificateJob ↔ artifact pipeline	Seal + cert PDF + archive; failures logged only
NotaryDigitalizationService	Re-runs ensureReady + separate blockchain anchoring

5. Migration Risks

Risk	Severity	Detail
Existing sealed documents	Critical	signature_hash tied to specific final PDF bytes; regeneration invalidates verification
Blockchain re-anchor	High	Hash refresh may replace transaction_id; old tx becomes stale
Root CA key migration	High	external://root-ca marker; rotating root affects new certs only
Signer keys in DB	High	Encrypted private keys; HSM move needs re-wrap or migration
Provider-managed rows	Medium	Nullable certificate_authority_id; chain uses issuer_certificate_pem
Partial seal state	High	signer_certificate_id set but signature_value null if job failed
Unique constraints	Medium	fingerprint_sha256 unique; re-issuance must handle collisions
Queued job ordering	Medium	GenerateDocumentPdfJob vs GenerateCertificateJob race

6. Signing Flow Risks

1. **Deferred cryptography** — UI shows signed after capture; PKI verify fails until job completes.
2. **One hash per signer** — Same document hash applied to all Signature rows for a signer.
3. **Legacy completeLegacySigning** — No field capture; may lack PKI signatures.
4. **Remote + local key mismatch** — Capture always creates local keys; remote seal uses provider keys.
5. **Trust authorization expiry** — Remote seal needs active session between capture and seal job.
6. **Silent job failure** — GenerateCertificateJob catches all errors; document may show Completed without seal.
7. **Cert issued before identity final** — DN from signer name/email at issuance; edits don't re-issue.
8. **DB-only revocation** — No CRL; external relying parties unaffected.

7. Blockchain Dependencies

Concern	Current behavior
What is anchored	SHA-256 of verifiable/final PDF only
When	During DocumentHashService.createOrRefreshForCompletedDocument (from sealing)
Failure mode	transaction_id = null; document still completes
Verification	Public verify uses verifyStoredProof; independent of PKI
Notary path	NotaryDigitalizationService may anchor again separately

Refactor note: Split hash computation, persistence, and anchoring before changing PKI logic.

8. Certificate Dependencies

Artifact	Created by	Stored	Used at verify
Root CA	CertificateAuthorityService	certificate_authorities	App-managed chain
Signer cert (app)	SignerCertificateService.issueForSigner	signer_certificates	Chain + RSA verify
Signer cert (provider)	createOrRefreshProviderManagedForSigner	+ issuer_certificate_pem	Provider chain
Seal binding	CompletedDocumentSealingService	signatures.* crypto cols	CertificateVerificationService
Completion cert PDF	DocumentCertificateService	documents.certificate_path	Download (not X.509)

9. Refactor Order (Phases 0–7)

Phase 0 — Observability (no behavior change)

- Structured logging at seal job boundaries
- Monitor: Completed documents with null signature_value
- Inventory production: certificate_source mix, null transaction_id

Phase 1 — Extract boundaries (interfaces, 1:1 adapters)

- CertificateIssuer, DocumentSealer, TrustAnchor, CertificateTrustValidator ports
- Existing classes remain default adapters

Phase 2 — Decouple capture from issuance

- Stop issuing certs in FieldSignatureCaptureService
- Move getOrIssueForSigner to seal time only

Phase 3 — Split hash / seal / anchor

- SealingService: seal only
- DocumentHashService: hash + persist only
- BlockchainAnchoringService: optional async job (feature flag, default off)

Phase 4 — Key store abstraction (HSM prep)

- Key handles vs raw PEM; dual-read for legacy rows

Phase 5 — External RCA / TSP adapter

- Map to provider_managed schema first
- app_managed root behind legacy flag

Phase 6 — Verification & revocation hardening

- Optional CRL/OCSP adapter; DB revocation remains UI source of truth

Phase 7 — Deprecate embedded root (long horizon)

- Freeze getOrCreateRootAuthority for new issuances after migration window
- Retain root rows for historical verification

10. High-Risk Areas

Area	Risk
CompletedDocumentSealingService.sealSignerSignatureSet	Re-seal overwrites rows; changes hash and blockchain proof
DocumentHashService.createOrRefreshForCompletedDocument	Hash refresh breaks public verify for old tx
CertificateAuthorityService.getOrCreateRootAuthority	Accidental second root or key rotation
SignerCertificate fingerprint unique	Re-issue collisions
GenerateCertificateJob silent catch	Masks PKI failures
Global signing_backend	Cannot migrate per tenant/document

11. Migration Strategy

Approach: Strangler fig + dual-write/read

1. Classify data: app_managed_sealed, app_managed_unsealed, provider_managed, legacy_no_fields
2. Introduce pki_backend resolution: signer → document → global (defaults unchanged)
3. New issuances on external RCA/TSP; retain app-managed for historical verify
4. Key migration in maintenance window: HSM for root; do not mass re-issue signer certs
5. Blockchain: never re-hash completed docs in place; version table if re-stamping required
6. Rollout: dev → staging snapshot → canary → cutover per signing_backend

Rollback

- Feature flags revert to current adapters
- PEM + certificate_authorities snapshot before HSM cutover
- Disable external issuer → fall back to embedded CA

12. Backward Compatibility Strategy

Surface	Contract to preserve
DB schema	Keep existing columns; add nullable only
certificate_source	app_managed / provider_managed
external://root-ca	File-backed root key indirection
Public verify	verifyDocumentSignatures + verifyStoredProof shapes
Encrypted signer keys	Laravel encrypted cast on signing_private_key
Routes	sign.show, sign.account.* unchanged during PKI refactor
Tests as contract	SignatureCertificateVerificationTest, CertificateAuthorityKeyStoreTest, PublicDocumentVerificationTest, SignerCertificateRevocationWorkflowTest

Compatibility rules

- Never change signature_hash algorithm without version field
- Never rotate root CA in place; add new CA row + superseded_by
- Provider path first for external RCA (issuer_certificate_pem)
- Defer CRL/SCEP/CMP until issuer adapter is stable

13. RCA Compliance Gap (Procurement Context)

Requirement	docuTrust today
EAL4+ / FIPS 140-2 L3 HSM RCA	Not implemented
Dedicated VGW + network isolation	Not implemented
CRL / CMP / SCEP / PKCS#7 CA services	Not implemented
Broad PKI interoperability (VPN, browsers, etc.)	Not implemented
Internal X.509 for DocuTrust documents	Partial (RSA 2048, app-managed)
Remote TSP via CSC	Partial (remote_managed path)

Recommendation: Deploy accredited commercial RCA on HSM; use docuTrust as relying party / enrollment client. Keep embedded OpenSSL root for dev/demo only.

Recommended first step

Execute **Phase 0 + Phase 1** only: monitoring for unsealed completed documents and interface extraction with zero delegation change.